

Verificación y Validación en un Sistema de Préstamo de Equipos de Laboratorio

Informe de avance

Renato Ramírez | Gonzalo Gutiérrez

Pruebas de Software

6 de septiembre de 2026

Repositorio: <https://github.com/xReNatS/Pruebas-de-software>

Índice

1. Introducción	3
1.1. Qué se construyó	3
2. Análisis y requerimiento mejorado	3
2.1. Ambigüedades, vacíos y riesgos detectados	3
2.2. Preguntas formuladas al cliente	4
2.3. Requerimiento mejorado	4
2.4. Requerimientos funcionales	5
3. Reglas de negocio, alcance y exclusiones	5
3.1. Reglas de negocio	5
3.2. Alcance	6
3.3. Exclusiones explícitas	6
4. Verificación y validación	6
4.1. La diferencia, aplicada a este proyecto	6
4.2. Actividades de verificación ejecutadas	6
4.3. Actividades de validación ejecutadas	7
4.4. Evidencia de la actividad Val2	9
5. Estados y diseño general	9
5.1. Estados de una solicitud	9
5.2. Transiciones automáticas	10
5.3. Cálculo de la disponibilidad	10
5.4. Arquitectura	10
6. Matriz de trazabilidad	11
7. Estrategia y resultados de pruebas	12
7.1. Objetivos y alcance	12
7.2. Responsabilidades y organización	12
7.3. Ambiente y datos de prueba	12
7.4. Criterios de entrada y de salida	13
7.5. Registro de evidencias	13
7.6. Resultados	13

8. Defectos, correcciones y reejecuciones	13
8.1. Defectos encontrados en la prueba cruzada	13
8.2. Cómo quedan vigilados	14
8.3. Defectos encontrados en el propio trabajo	14
9. Trabajo colaborativo	15
9.1. Flujo de trabajo con Git	15
9.2. Convenciones	15
9.3. Evidencia de cambios originados por una revisión	15
9.4. Manejo de secretos	15
10. Uso de inteligencia artificial	15
10.1. Herramientas y actividades	15
10.2. Dos ejemplos de prompts	15
10.3. Resultados aceptados, modificados y descartados	16
10.4. Cómo se verificaron las respuestas	16
10.5. Decisiones que no se delegaron	16
10.6. Limitación o recomendación dudosa detectada	16
11. Reflexiones individuales y conclusiones	17
11.1. Reflexión de Renato	17
11.2. Reflexión de Gonzalo	17
11.3. Conclusiones	18
11.4. Riesgos abiertos	18

1. Introducción

Un laboratorio universitario presta equipos tecnológicos a estudiantes y profesores. Hoy las solicitudes llegan por correo o mensajería instantánea y la disponibilidad se controla con una planilla que no siempre está actualizada. El encargado pidió una solución *simple, confiable, que evite problemas con los préstamos*, sin entregar más detalle.

Ese enunciado no es un requerimiento: es un deseo. No dice qué datos definen a una persona ni a un equipo, no fija los estados de un préstamo ni quién puede moverlos, no acota la duración ni la cantidad, y no explica qué significa que un equipo esté disponible. El primer trabajo del equipo fue convertirlo en algo que se pudiera construir y, sobre todo, comprobar.

Este informe documenta ese proceso: las ambigüedades detectadas, las decisiones tomadas para cerrarlas, las reglas de negocio que resultaron, la aplicación construida y, con más peso, las actividades de verificación y validación ejecutadas sobre ella.

1.1. Qué se construyó

Una aplicación de línea de comando en Python, con persistencia en archivos JSON, autenticación con dos roles, registro de eventos e integración con Sentry para el reporte de defectos. La tecnología es de libre elección y se prefirió Python por familiaridad del equipo y por la disponibilidad de `pytest`.

2. Análisis y requerimiento mejorado

2.1. Ambigüedades, vacíos y riesgos detectados

El enunciado se sometió a una lectura crítica buscando todo aquello que dos personas podrían interpretar de forma distinta. Se identificaron doce puntos, por encima del mínimo de ocho exigido. Cada uno se resolvió con una decisión explícita, y esa decisión está reflejada en el código y en al menos un caso de prueba.

ID	Ambigüedad	Decisión tomada
A01	El enunciado no define los datos obligatorios de cada entidad	Se fijó el esquema de las cuatro entidades. Riesgo aceptado: no se guarda carrera ni departamento, así que no habrá reportes por unidad académica
A02	¿Un equipo es un modelo o una unidad física?	Cada registro es una unidad física con código propio. Es lo que hace que la disponibilidad sea una pregunta binaria por unidad y no un conteo de existencias
A03	Contradicción interna: el período de gracia aparecía como 1 día en las reglas y como 2 días en la lista de dudas	Se adoptó 1 día, tomado de la sección de reglas por ser la más formal. Vive en una constante, así que cambiarlo es modificar un número y volver a ejecutar la suite
A04	¿Un encargado puede además pedir equipos?	No. Las cuentas son separadas y un mismo correo no puede existir en las dos colecciones
A05	¿Los estados <i>aprobada</i> y <i>en préstamo</i> ocupan igual el equipo?	Sí, y además <i>por revisar</i> también ocupa. Un equipo queda bloqueado desde que alguien lo pide hasta que la solicitud se cierra

ID	Ambigüedad	Decisión tomada
A06	¿Qué pasa si dos personas piden la misma unidad a la vez?	Gana quien registre primero. Como la solicitud bloquea desde <i>por revisar</i> , la segunda persona recibe un error que nombra la solicitud que la tiene tomada
A07	RF08 y RF10 son dos requerimientos para un solo hecho físico	El solicitante <i>declara</i> la devolución; el encargado la <i>confirma</i> . Solo la confirmación libera la unidad
A08	¿Se puede modificar una solicitud ya aprobada?	No se edita. Se cancela y se crea otra, con motivo obligatorio que queda en el historial
A09	¿Qué significa <i>confiable</i> para el encargado?	Se tradujo a tres propiedades verificables (ver sección 2.3)
A10	¿Cuál es la anticipación máxima de una reserva?	No hay límite hacia el futuro, porque el bloqueo es inmediato de todas formas. Lo que sí se limita es la duración
A11	¿Estudiantes y profesores tienen los mismos límites?	Sí. El modelo no distingue subtipos de solicitante
A12	El estado del equipo podría desincronizarse de las solicitudes	El único valor manual es <i>fuera de servicio</i> . Los demás se calculan leyendo las solicitudes activas

Cuadro 1: Ambigüedades detectadas y decisiones tomadas.

2.2. Preguntas formuladas al cliente

Las preguntas se organizaron por área y se plantearon en la fase de análisis: datos obligatorios de cada entidad; estados posibles de un préstamo y quién ejecuta cada transición; duración máxima y anticipación permitida; consecuencias de no devolver a tiempo; cantidad máxima de equipos por persona; si se pueden tener varias solicitudes activas; qué significa exactamente que un equipo esté disponible; si una reserva futura bloquea desde ya; y qué puede y qué no puede hacer cada rol.

2.3. Requerimiento mejorado

La frase *simple, confiable, que evite problemas con los préstamos* se tradujo a tres propiedades que sí se pueden comprobar:

1. **Ninguna operación puede dejar un archivo a medio escribir.** La escritura es atómica: se escribe un temporal y se reemplaza.
2. **Ninguna transición de estado puede ocurrir fuera de la tabla de transiciones.** Toda operación que cambia el estado de una solicitud pasa por la misma función, así que no existe camino en el código capaz de saltársela.
3. **Todo evento relevante queda registrado con su actor y su marca de tiempo.**

Las tres tienen casos de prueba asociados, así que *confiable* dejó de ser un adjetivo y pasó a ser algo que se aprueba o se reprueba.

2.4. Requerimientos funcionales

ID	Requerimiento	Rol	Estado
RF01	Inicio de sesión	Ambos	Implementado
RF02	Registrar persona autorizada	Encargado	Implementado
RF03	Registrar equipo	Encargado	Implementado
RF04	Consultar equipo	Ambos	Implementado
RF05	Crear solicitud	Solicitante	Implementado
RF06	Consultar solicitudes	Ambos	Implementado
RF07	Aprobar o rechazar solicitud	Encargado	Implementado
RF08	Registrar entrega y confirmar devolución	Encargado	Implementado
RF09	Cancelar solicitud	Solicitante	Implementado
RF10	Declarar devolución	Solicitante	Implementado
RF11	Renovar el préstamo	Solicitante	Implementado

Cuadro 2: Requerimientos funcionales y su estado al cierre de este avance.

3. Reglas de negocio, alcance y exclusiones

3.1. Reglas de negocio

Cada regla tiene identificador estable, usado en el código, en las pruebas y en los *issues*. Las constantes viven en un único módulo, de modo que las pruebas las referencian en vez de repetir números.

ID	Regla	Verificada
RN01	Un solicitante puede tener como máximo 2 equipos simultáneamente	CP-50
RN02	Los equipos en poder de una persona deben ser de categorías distintas	CP-26, CP-51
RN03	Una solicitud incluye 1 o 2 equipos	CP-27
RN04	La duración máxima de un préstamo es de 1 semana	CP-28, CP-53
RN05	Tras la aprobación hay 2 días para retirar; si no, se cancela	Pendiente
RN06	Hay 1 día de gracia tras el vencimiento antes de marcar atraso	Pendiente
RN07	Se permite una sola renovación, de hasta 1 semana	CP-36
RN08	Un correo es solicitante o encargado, nunca ambos	Verificación manual
RN09	Solo los solicitantes generan solicitudes	CP-25
RN10	Con 2 equipos en su poder no se pueden crear nuevas solicitudes	CP-31
RN11	Quien no devuelve a tiempo queda <i>pendiente</i> y no puede pedir más	CP-30, CP-37

ID	Regla	Verificada
RN12	Un equipo tomado por una solicitud activa no se puede reservar	CP-10 a CP-13, CP-54
RN13	El solicitante puede devolver antes de tiempo	CP-35
RN14	Estudiantes y profesores tienen los mismos límites	Por diseño del modelo
RN15	El estado <i>atrasada</i> lo calcula el sistema, nunca una persona	CP-06 a CP-09

Cuadro 3: Reglas de negocio y el caso de prueba que las verifica.

3.2. Alcance

Registro y consulta de personas autorizadas y de equipos; autenticación con dos roles; creación de solicitudes de uno o dos equipos; aprobación y rechazo; registro de entregas, devoluciones y cancelaciones; consulta de préstamos vigentes, futuros y atrasados; cálculo de disponibilidad; persistencia en JSON; log de eventos; y datos de demostración.

3.3. Exclusiones explícitas

Se dejan fuera de forma deliberada: multas, cobros o sanciones económicas; historial de mantenimiento o reparación; interfaz gráfica o web; autorregistro de usuarios y recuperación de contraseña; gestión física de inventario por ubicación o sede; y notificaciones por correo o mensajería.

Se excluye además la reserva por rango de fechas con liberación automática: como un préstamo puede extenderse, el sistema no puede prometer que una unidad se desocupe en una fecha futura.

4. Verificación y validación

4.1. La diferencia, aplicada a este proyecto

Verificar es preguntarse si se está construyendo correctamente el producto especificado. En este proyecto significa comprobar que el código respeta la tabla de transiciones acordada, que las constantes de negocio son las que el equipo definió, y que cada requerimiento cumple sus criterios de aceptación. Es una comparación contra un documento propio.

Validar es preguntarse si se está construyendo el producto que realmente se necesita. Aquí significa comprobar que las reglas que se especificaron resuelven el problema del encargado y no otro. Es una comparación contra la realidad, y por eso es la que puede invalidar la especificación misma.

La diferencia se hizo concreta en este proyecto de una forma que conviene destacar. La verificación de RF05 fue exitosa: sus ocho casos de aceptación pasaron. La validación encontró que el requerimiento estaba *bien construido pero mal especificado*: cada caso validaba una solicitud aislada, mientras que las reglas del enunciado hablan de lo que una persona tiene en su poder. Se estaba construyendo correctamente un producto que no era el que se necesitaba.

4.2. Actividades de verificación ejecutadas

ID	Actividad
V1	Contraste de la tabla de transiciones contra el diagrama acordado. <i>Objetivo:</i> comprobar que el código implementa exactamente las transiciones definidas, ni una más ni una menos. <i>Responsable:</i> Renato. <i>Evidencia:</i> casos CP-06 a CP-09, que recorren las transiciones válidas, las inexistentes, las ejecutadas por un rol no autorizado y los estados finales. <i>Resultado:</i> 18 ejecuciones aprobadas. <i>Conclusión:</i> la máquina de estados es fiel al diseño. Se separaron deliberadamente dos errores distintos, transición inexistente y transición sin permiso, porque diagnostican problemas diferentes.
V2	Ejecución de la suite completa antes de abrir cada pull request. <i>Objetivo:</i> que ningún cambio llegue a la rama principal dejándola en rojo. <i>Responsable:</i> quien abre el <i>pull request</i>. <i>Evidencia:</i> el estado de la suite se declara en el cuerpo de cada <i>pull request</i> y en el mensaje de <i>commit</i>. <i>Resultado:</i> los 6 <i>pull requests</i> fusionados registran la suite en verde. <i>Conclusión:</i> el criterio de salida se cumplió en todos los casos.
V3	Revisión cruzada de código en pull requests. <i>Objetivo:</i> que ningún cambio se integre sin que la otra persona lo lea. <i>Responsable:</i> el integrante que no escribió el código. <i>Evidencia:</i> PR #26, con revisión detallada que originó tres <i>issues</i> de defecto. <i>Resultado:</i> la revisión encontró tres defectos que la suite del autor no detectaba. <i>Conclusión:</i> la revisión cruzada aportó más que la ejecución de pruebas del propio autor, porque cuestionó el supuesto y no solo el código.
V4	Comprobación de que las contraseñas no se almacenan en texto plano. <i>Objetivo:</i> verificar una propiedad de seguridad afirmada en el diseño. <i>Responsable:</i> Renato. <i>Evidencia:</i> caso CP-05, más una búsqueda directa de la contraseña de demostración sobre todos los archivos JSON. <i>Resultado:</i> el JSON solo contiene el campo <code>contrasena_hash</code> con prefijo <code>pbkdf2_sha256\$</code>. La búsqueda no devuelve coincidencias. <i>Conclusión:</i> la propiedad se cumple y queda vigilada por un caso.
V5	Anclaje del orden de los menús al guion de demostración. <i>Objetivo:</i> evitar que el guion cite números de opción que ya no corresponden. <i>Responsable:</i> Renato. <i>Evidencia:</i> caso CP-44, que compara el orden real de las opciones con el que el guion documenta. <i>Resultado:</i> aprobado. <i>Conclusión:</i> un documento que cita identificadores volátiles necesita algo que lo vigile, o envejece en silencio.

Cuadro 4: Actividades de verificación ejecutadas.

4.3. Actividades de validación ejecutadas

ID	Actividad
Val1	Ensayo completo del guion de demostración. <i>Objetivo:</i> comprobar que un tercero puede reproducir la demostración siguiendo solo el documento. <i>Responsable:</i> Renato. <i>Evidencia:</i> ejecución automatizada del guion paso a paso, comprobando la respuesta de cada uno. <i>Resultado:</i> falló. En tres pasos el guion no indicaba qué responder a todos los campos, porque la aplicación pide los cuatro datos de una persona antes de validar ninguno. <i>Conclusión:</i> el documento era correcto para quien ya conocía el sistema e inservible para quien no. Se corrigió y se volvió a ensayar. Es el tipo de defecto que solo aparece al validar, nunca al verificar.
Val2	Verificación de la integración con Sentry contra el panel real. <i>Objetivo:</i> comprobar que un defecto real llega al panel con información suficiente para diagnosticarlo sin pedirle nada al usuario. <i>Responsable:</i> Renato. <i>Evidencia:</i> <i>issue</i> 7715896391 en Sentry, provocado truncando a propósito el archivo de equipos. <i>Resultado:</i> el evento llega con la traza completa, las variables locales de cada marco, el rastro de la sesión y el <i>commit</i> exacto. <i>Conclusión:</i> la integración cumple su propósito. El proceso destapó tres defectos propios, documentados en la sección 8.
Val3	Prueba cruzada de RF05 combinando reglas entre solicitudes. <i>Objetivo:</i> comprobar que las reglas del enunciado se cumplen en la realidad y no solo dentro de una operación. <i>Responsable:</i> Renato, sobre código de Gonzalo. <i>Evidencia:</i> casos CP-50 a CP-55. <i>Resultado:</i> falló. Se consiguió que una persona quedara con tres solicitudes activas y cinco equipos comprometidos, y con dos equipos de la misma categoría. <i>Conclusión:</i> la implementación era correcta respecto de sus criterios de aceptación, pero los criterios estaban escritos sobre una solicitud aislada mientras la regla habla de una persona. Es el hallazgo más valioso del avance.
Val4	Contraste de las reglas implementadas contra las respuestas del cliente. <i>Objetivo:</i> comprobar que ninguna regla del código contradice lo acordado en la fase de análisis. <i>Responsable:</i> ambos integrantes. <i>Evidencia:</i> tabla de reglas RN01 a RN15 con su caso asociado. <i>Resultado:</i> se detectó una contradicción interna en el material de origen, que daba el período de gracia como 1 día y como 2 días. <i>Conclusión:</i> se resolvió por decisión documentada (A03) en vez de elegir en silencio, que es lo que habría ocurrido si la contradicción no se busca.
Val5	Revisión de que el escenario de demostración permite mostrar cada estado sin esperar días reales. <i>Objetivo:</i> comprobar que el sistema se puede demostrar y probar sin depender del paso del tiempo. <i>Responsable:</i> Renato. <i>Evidencia:</i> casos CP-45, CP-46 y CP-47. <i>Resultado:</i> faltaba el estado <i>cancelada</i>; el escenario cubría siete de los ocho estados. CP-46 comprueba además que el solicitante bloqueado tenga una solicitud atrasada que lo explique, y no un estado escrito a mano. <i>Conclusión:</i> corregido. Un dato de demostración sin respaldo en el resto del escenario es una demostración que miente.

Cuadro 5: Actividades de validación ejecutadas.

4.4. Evidencia de la actividad Val2

```
54     Un ErrorDominio es una situacion prevista y se muestra al usuario. Un
55     NotImplementedError marca una funcionalidad todavia no construida. Todo
56     lo demas es un defecto: se registra en el log y se envia a Sentry.
57     ---
58     try:
59         return accion()
60     except ErrorDominio as fallo:
61         error(str(fallo))
62     except NotImplementedError as pendiente:
63         aviso(f"Funcionalidad pendiente ({pendiente}). Ver los Issues del repositorio.")
64     except Exception as fallo:
65
66 accion      <function _despachar.<locals>.<lambda> at 0x0000026215398360>
67 actor      "ENC-0001"
68 fallo      JSONDecodeError('Expecting property name enclosed in double quotes: line 1 column 23 (char 22)')
```

Figura 1: Detalle del evento en Sentry. Se ven el marco de la aplicación donde se atrapó la excepción y las variables locales de ese marco, incluido el actor que ejecutaba la operación. Es lo que permite diagnosticar un defecto sin pedirle al usuario que lo reproduzca.

Observación. Las cinco actividades de verificación resultaron aprobadas. Tres de las cinco de validación encontraron problemas. No es casualidad: verificar compara el producto contra lo que el equipo escribió, y el equipo tiende a escribir lo que ya pensó. Validar lo compara contra lo que hace falta, y ahí aparecen los supuestos que nadie cuestionó.

5. Estados y diseño general

5.1. Estados de una solicitud

Origen	Acción	Destino	Roles	Impedimentos
por_revisar	aprobar	aprobada	Encargado	El equipo dejó de estar disponible
por_revisar	rechazar	rechazada	Encargado	Motivo vacío
por_revisar	cancelar	cancelada	Solicitante, Encargado	No es su solicitud
aprobada	entregar	en_prestamo	Encargado	Pasaron más de 2 días
aprobada	cancelar	cancelada	Solicitante, Encargado, Sistema	—
en_prestamo	vencer	periodo_gracia	Sistema	No llega la fecha
en_prestamo	concluir	concluida	Encargado	—
periodo_gracia	renovar	en_prestamo	Solicitante, Encargado	Ya usó su renovación
periodo_gracia	concluir	concluida	Encargado	—
periodo_gracia	atrasar	atrasada	Sistema	No vence la gracia
atrasada	concluir	concluida	Encargado	—

Origen	Acción	Destino	Roles	Impedimentos
--------	--------	---------	-------	--------------

Cuadro 6: Transiciones permitidas, con el rol autorizado y las condiciones que impiden cada operación.

Los estados *rechazada*, *cancelada* y *concluida* son finales. El caso CP-09 comprueba que no admiten ninguna acción.

5.2. Transiciones automáticas

Las ejecuta el sistema con rol propio, y ninguna persona puede dispararlas a mano. Al pasar la fecha de devolución, *en préstamo* pasa a *período de gracia*. Al vencer el día de gracia, pasa a *atrasada* y el solicitante queda en estado *pendiente*. Si transcurren más de dos días desde la aprobación sin que se registre la entrega, la solicitud se cancela.

5.3. Cálculo de la disponibilidad

Un equipo está disponible cuando no está marcado fuera de servicio y ninguna solicitud en estado activo lo incluye. Los estados activos son *por revisar*, *aprobada*, *en préstamo*, *período de gracia* y *atrasada*.

Tres consecuencias buscadas de esta definición:

- Una solicitud sin aprobar ya bloquea el equipo, lo que resuelve la carrera entre dos personas que piden la misma unidad.
- No hay disponibilidad por rango de fechas, porque un préstamo puede extenderse y el sistema no puede prometer una fecha de liberación.
- El estado del equipo se deriva, no se duplica. En el archivo una unidad prestada sigue diciendo *disponible*; es la consulta la que muestra *en uso*. Así no puede quedar desincronizado.

5.4. Arquitectura

Cuatro capas, con una regla: cada una solo conoce la de abajo.

Capa	Responsabilidad
Interfaz	Menús por rol. Traduce errores del dominio a mensajes y captura lo inesperado sin dejar caer la aplicación
Servicios	Un módulo por área funcional. Aplica permisos, validaciones y reglas de negocio
Dominio	Máquina de estados, cálculo de disponibilidad, entidades, validaciones y seguridad
Persistencia	Lectura y escritura atómica de los archivos JSON

El reparto del trabajo siguió esta división por dominio funcional y no por capa, para que cada integrante fuera dueño de archivos distintos y los conflictos de integración fueran raros.

6. Matriz de trazabilidad

Los identificadores son los mismos en este documento, en el código, en las pruebas y en los *issues*.

ID	Criterio de aceptación	Casos	Resultado
RF01	Credenciales válidas dan una sesión con el rol correcto; las inválidas dan un mensaje que no revela si el correo existe	CP-01 a CP-05	Aprobado
RF02	Solo un encargado registra personas; RUT y correo únicos; contraseña como hash	CP-16 a CP-18	Aprobado, cobertura parcial
RF03	Solo un encargado registra equipos; código único; dos unidades del mismo modelo conviven	CP-19 a CP-22	Aprobado, cobertura parcial
RF04	El solicitante ve solo disponibles; el encargado ve todos con estado calculado	CP-23, CP-13	Aprobado, cobertura parcial
RF05	Nace en <i>por revisar</i> , 1 o 2 equipos de categorías distintas, hasta 7 días, sobre equipos disponibles	CP-24 a CP-31, CP-53 a CP-55	Aprobado, con defectos #27, #28, #29
RF06 a RF11	Ver sección de requerimientos	CP-32 a CP-38	Pendientes
RN01	No más de 2 equipos simultáneos sumando todas las solicitudes activas	CP-50	Fallido , #27
RN02	Categorías distintas aunque los equipos vengan de solicitudes separadas	CP-51	Fallido , #28
RN12	Toda solicitud activa bloquea sus equipos	CP-10 a CP-13, CP-54	Aprobado
RN15	Transiciones fuera de la tabla o con rol no autorizado fallan	CP-06 a CP-09	Aprobado
VAL01	RUT con dígito verificador incorrecto rechazado antes de tocar el disco	CP-14, CP-15	Aprobado
INF01	Todo evento queda en el log con su actor; un defecto no tumba la aplicación	CP-40, CP-41, CP-43	Aprobado
INF02	Los errores previstos no van a Sentry y la suite no envía eventos reales	CP-39, CP-42	Aprobado
INF03	Un defecto real llega a Sentry con traza, variables y el <i>commit</i>	Verificación manual	Aprobado

ID	Criterio de aceptación	Casos	Resultado
DEM01	El escenario incluye un ejemplo de cada estado y un solicitante bloqueado	CP-45 a CP-47	Aprobado
DEM02	La demostración es reproducible y el guion no se desactualiza en silencio	CP-44, CP-48, CP-49	Aprobado

Cuadro 7: Matriz de trazabilidad. Se trazan 16 requerimientos, por sobre el mínimo de 10.

Nota sobre la cobertura parcial. En RF02, RF03 y RF04 varios criterios quedaron sin caso automatizado de forma deliberada: su diseño corresponde a la prueba cruzada del otro integrante (#14), y escribirlos uno mismo habría anulado el propósito de esa prueba. Se comprobaron de forma manual y todos se cumplen, pero mientras no exista el caso, esos requerimientos no se declaran cerrados.

7. Estrategia y resultados de pruebas

7.1. Objetivos y alcance

El objetivo es comprobar que el sistema respeta las reglas de negocio y la máquina de estados, y que ninguna entrada inválida deja los archivos en un estado inconsistente.

Entra en el alcance la capa de servicios, la máquina de estados, el cálculo de disponibilidad, las validaciones y la persistencia. Queda fuera la interacción por teclado, que se verifica de forma manual con el guion de demostración, y la integración real con Sentry, comprobada una vez de forma manual.

7.2. Responsabilidades y organización

Las pruebas están cruzadas: cada integrante diseña y ejecuta los casos que verifican el código de la otra persona. Quien encuentra un defecto lo registra como *issue*; quien escribió el código lo corrige; y la reejecución la hace quien lo reportó, no quien lo corrigió.

Integrante	Implementa	Prueba
Renato	RF01 a RF04, infraestructura	RF05 a RF11
Gonzalo	RF05 a RF11	RF01 a RF04

7.3. Ambiente y datos de prueba

Python 3.11 o superior con `pytest` 8. Cada caso corre contra un directorio de datos temporal, así que la suite se puede ejecutar en cualquier orden y no ensucia los datos de demostración. Sentry queda desactivado durante las pruebas, lo que a su vez está vigilado por el caso CP-39.

Los escenarios que requieren tiempo transcurrido se montan insertando solicitudes ya en el estado deseado, de modo que no hay que esperar días reales ni depender de código todavía no implementado.

7.4. Criterios de entrada y de salida

Para entrar: el requerimiento tiene criterio de aceptación escrito en la matriz y la suite corre sin errores de importación. Para salir: todos sus casos pasan, ningún caso queda marcado como pendiente para un requerimiento que se declara terminado, y los defectos de severidad alta están cerrados y reejecutados.

7.5. Registro de evidencias

La planilla de casos de prueba **se genera, no se escribe a mano**. Un script recorre los archivos de prueba, extrae identificador, categoría, precondiciones y resultado esperado de cada caso, y lo cruza con el reporte de una ejecución real de la suite.

La razón es directa: el enunciado considera incompleto un caso que solo tenga el resultado esperado. Si el resultado obtenido se escribe a mano, la planilla y el código dejan de coincidir en cuanto alguien toca una prueba, y nadie se entera.

El generador distingue tres situaciones que escritas a mano se confunden: aprobado; defecto abierto, citando el número del *issue* que lo registra; y pendiente de que exista su requerimiento.

7.6. Resultados

Categoría	Mínimo exigido	Ejecutados
Funcionales	5	13
De borde	4	11
Negativos o con entradas inválidas	3	19
Que combinan reglas	2	11
Escenario completo	1	1
Total de casos	15	55

Un caso puede pertenecer a más de una categoría, por lo que la suma por categoría supera el total.

Estado	Casos
Aprobados	55
Con defecto abierto	0
A la espera de su requerimiento	0
Fallidos sin explicación	0

El escenario completo. El caso CP-38 recorre el flujo desde crear la solicitud hasta liberar el equipo, cubriendo RF05, RF07, RF08 y RF10 en un solo hilo.

8. Defectos, correcciones y reejecuciones

8.1. Defectos encontrados en la prueba cruzada

Los tres se encontraron revisando RF05, y los tres vienen del mismo malentendido: los criterios de aceptación estaban escritos sobre una solicitud aislada, mientras que las reglas del enunciado hablan de lo que una persona tiene en su poder. Por eso los ocho casos de aceptación del requerimiento no los detectaban.

Issue	Severidad	Descripción
#27	Alta	Partir el pedido en varias solicitudes salta el tope de dos equipos. Se reprodujo dejando a una persona con tres solicitudes activas y cinco equipos comprometidos. El recuento usa solo los estados en que el equipo ya está entregado, así que una solicitud por revisar o aprobada no cuenta
#28	Alta	Pedir dos laptops en una misma solicitud se rechaza; pedir las en dos solicitudes seguidas, no. El resultado físico es idéntico
#29	Media	Se acepta una solicitud con fechas ya pasadas. Nace vencida y además inmoviliza el equipo, porque toda solicitud activa lo bloquea

Cuadro 8: Defectos registrados durante la prueba cruzada de RF05.

Justificación de la severidad. Los dos primeros son altos porque rompen reglas centrales del enunciado y no requieren ninguna maniobra deliberada: cualquier solicitante los consigue sin proponérselo, pidiendo en dos veces en vez de una. El tercero es medio porque no rompe una regla escrita, pero deja el sistema en un estado sin sentido e inmoviliza una unidad hasta que alguien intervenga.

8.2. Cómo quedan vigilados

Cada defecto tiene su caso de prueba escrito, marcado de forma que hoy falla como se espera. Cuando el defecto se corrija, el caso pasará y la marca hará fallar la suite, lo que obliga a retirarla. Un defecto corregido no puede quedarse en silencio.

8.3. Defectos encontrados en el propio trabajo

La verificación de la integración con Sentry destapó tres problemas que no se habrían visto sin probar de punta a punta:

1. **Los eventos no llegaban con identificador.** La integración de *logging* del SDK creaba su propio evento antes que el explícito, y la deduplicación descartaba el segundo. Corregido desactivando la creación automática.
2. **El registro de eventos quedaba fijado al primer directorio del proceso.** Hacía imposible verificar el log de forma aislada. Corregido.
3. **La suite completa se estaba conectando a Sentry con el DSN real.** El valor se leía una sola vez al importar el módulo, así que la desactivación de las pruebas llegaba tarde y cada ejecución habría ensuciado el panel con eventos falsos. Corregido, y vigilado por el caso CP-39.

A esto se suma el defecto encontrado en el guion de demostración (Val1), que no era un defecto del código sino del documento, y que habría arruinado una demostración en vivo.

9. Trabajo colaborativo

9.1. Flujo de trabajo con Git

Se eligió **trunk-based con ramas cortas**. La rama principal es la única de larga vida y siempre debe quedar en verde. Todo lo demás son ramas que nacen de ella, viven uno o dos días y vuelven por *pull request*.

Justificación. GitFlow separa ramas de desarrollo, publicación y corrección para coordinar varias versiones publicadas al mismo tiempo. Este proyecto tiene dos personas, una entrega y ninguna versión en producción, así que esas ramas serían ceremonia sin beneficio. Trunk-based mantiene la integración diaria, que es lo que de verdad reduce el riesgo de conflictos grandes al final del plazo.

9.2. Convenciones

Las ramas llevan el identificador del requerimiento en el nombre, lo que las conecta con el *issue*, con el caso de prueba y con la fila de la matriz. Los mensajes de *commit* nombran el requerimiento en la primera línea y explican en el cuerpo por qué se hizo el cambio, no solo qué cambió.

9.3. Evidencia de cambios originados por una revisión

La revisión del PR #26 originó los *issues* #27, #28 y #29, cada uno con su caso de prueba. Es evidencia directa de que la revisión cruzada produjo cambios y no fue un trámite.

Una segunda evidencia, más incómoda pero más honesta: al reportar esos defectos se citaron los casos de prueba por su identificador, pero esos casos vivían en una rama que el otro integrante no tenía a mano. El reporte era correcto e inútil. Se corrigió incorporando a cada *issue* el código completo del caso y la salida real del fallo.

9.4. Manejo de secretos

El repositorio no contiene credenciales. El DSN de Sentry vive en un archivo de entorno excluido del control de versiones. Durante el avance el DSN se colocó por error en el archivo de ejemplo, que sí se versiona; se detectó antes de que llegara a ningún *commit* y se movió al lugar correcto.

10. Uso de inteligencia artificial

10.1. Herramientas y actividades

Se utilizó **Claude** de Anthropic, en dos modalidades: la interfaz conversacional para el análisis del requerimiento y la discusión de decisiones de diseño, y **Claude Code** para la implementación, la ejecución de pruebas y la gestión del repositorio.

Actividades en las que se usó: propuesta de estructura del proyecto; redacción de la implementación a partir de decisiones ya tomadas por el equipo; diseño y ejecución de casos de prueba; redacción de documentación; y análisis del código del otro integrante durante la prueba cruzada.

10.2. Dos ejemplos de prompts

Ejemplo 1, fase de análisis. Se entregó a la herramienta el conjunto completo de decisiones ya cerradas por el equipo, indicando explícitamente qué puntos seguían abiertos y pidiendo que no asumiera nada sobre ellos:

Estoy desarrollando junto a un compañero una aplicación de línea de comandos para un sistema de préstamo de equipos [...]. Ya hicimos el análisis del requerimiento; a continuación te doy todas las decisiones tomadas. Quiero que me ayudes a implementarla de forma incremental, explicándome las decisiones de diseño y sin dar por sentado nada que yo no haya especificado.

1. ¿El período de gracia dura 1 o 2 días? (aún no lo cerramos). 2. ¿Un encargado puede ser también solicitante, o son cuentas separadas? 3. ¿Los estados `.aprobada` `.entregada` `.ocupan` igual el equipo? 4. RF08 y RF10 son los dos lados de la devolución; confírmame cómo modelarlos.

El resultado fue que la herramienta preguntó por esos cuatro puntos en vez de inventarlos, y el equipo los resolvió antes de que se escribiera código.

Ejemplo 2, prueba cruzada. Al llegar el *pull request* del otro integrante, la instrucción fue deliberadamente abierta:

```
hay un pr de mi compañero haz las pruebas necesarias
```

De ahí salió la detección de los tres defectos. El punto relevante es que no se pidió *aprobar* el *pull request* sino *probarlo*, lo que orientó el trabajo a buscar fallas y no a confirmar que estaba bien.

10.3. Resultados aceptados, modificados y descartados

Aceptados: la estructura de carpetas por capas; la idea de derivar el estado del equipo en vez de guardarlo duplicado; el uso de una tabla de transiciones única como punto de paso obligatorio.

Modificados: la propuesta inicial de la planilla de casos de prueba era una tabla escrita a mano; el equipo la reemplazó por un generador, porque una planilla escrita a mano se desincroniza del código. La verificación de RF02 se propuso inicialmente con casos automatizados propios; se cambió a verificación manual para no anular la prueba cruzada del compañero.

Descartados: la implementación de un único archivo JSON para manejar todas las entidades del problema. Se optó por utilizar un archivo separado para cada una.

10.4. Cómo se verificaron las respuestas

Ninguna afirmación se aceptó sin ejecutarla. Cada implementación se comprobó corriendo la suite; la integración con Sentry se verificó contra el panel real provocando un defecto; el guion de demostración se ensayó paso a paso, y falló; y los tres defectos reportados al compañero se reprodujeron antes de escribir el reporte.

10.5. Decisiones que no se delegaron

Las doce decisiones de la tabla de ambigüedades las tomó el equipo. En particular, la contradicción del período de gracia (A03) se resolvió eligiendo la fuente más formal, y la decisión de que una solicitud sin aprobar ya bloquee el equipo (A05) se tomó pesando la pérdida de utilización del laboratorio contra el riesgo de prometer una fecha de liberación imposible de cumplir. Tampoco se delegaron el alcance ni las exclusiones.

10.6. Limitación o recomendación dudosa detectada

Un caso concreto observado: al implementar la integración con Sentry, la herramienta produjo código que parecía correcto y que efectivamente registraba el error, pero que no obtenía el identificador del evento porque la biblioteca ya había creado uno por su cuenta desde el sistema de *logging*. El problema solo apareció al probar contra el panel real. La lección es que el código generado puede ser plausible y funcionar en apariencia, y que la única forma de saberlo es ejecutarlo contra el sistema real, no leerlo.

11. Reflexiones individuales y conclusiones

11.1. Reflexión de Renato

1. ¿Qué ambigüedad influyó más en el diseño y cómo la resolvieron? La ambigüedad más compleja de abordar fue A05: si el bloqueo de una unidad ocurre desde que se solicita o sólo cuando es aprobada. Como equipo, decidimos asegurar que los solicitantes tuvieran vistas que reflejaran el flujo promedio en donde sí se retiran los equipos, para de esta forma no publicar cosas de las que no se tiene certezas. Esta decisión determinó como se hace el cálculo de disponibilidad y los criterios de aceptación correspondientes.
2. ¿Cuál fue la prueba más útil y qué riesgo permitió reducir? La prueba más útil fue la prueba cruzada sobre RF05, que reveló tres defectos (issues #27, #28, #29) que los ocho criterios de aceptación no detectaban inicialmente. Estos demostraron que los criterios de aceptación no definían por completo las reglas del negocio y permitía a una persona tener más de un equipo de la misma categoría a la vez. Esto nos ayudó a implementar correctamente el código que respalda la regla de negocio relacionando a la cantidad máxima de equipos en posesión por solicitante.
3. ¿Qué fallo o comportamiento inesperado detectaron y cómo lo corrigieron? Durante la integración con Sentry, encontramos tres problemas inesperados: en primer lugar, el SDK de logging creaba un evento automático antes de que lo enviáramos explícitamente, y como eran parecidos, la deduplicación descartaba el nuestro. En segundo lugar, el registro de eventos siempre iba al primer directorio del proceso, lo que impedía verificarlo de forma aislada. Por último, la suite de pruebas se conectaba al DSN real de Sentry al importar el módulo, ensuciando el panel con eventos falsos. Cada uno se corrigió ajustando la configuración: desactivando la creación automática, haciendo que el registro fuera relativo al directorio del proyecto y usando un DSN de prueba aislado.
4. ¿Qué aporte de su compañero mejoró su trabajo? Incluir evidencia. Mi compañero Gonzalo implementó las correcciones de los defectos #27, #28 y #29 que yo reporté en la prueba cruzada de RF05. La evidencia está en los pull requests del repositorio, donde las correcciones se integraron a través de revisiones que pude validar antes de reejecutar los casos CP-50 a CP-52. Sin ese trabajo de depuración, las reejecuciones no habrían sido independientes y el ciclo de verificación cruzada habría quedado incompleto.
5. ¿Qué parte del sistema podría fallar todavía? Persiste el riesgo de concurrencia: al ser la aplicación de un solo proceso y no bloquear archivos, dos instancias sobre el mismo directorio de datos podrían pisarse si se ejecutan simultáneamente.
6. ¿Qué aprendieron sobre la diferencia entre verificar y validar? Aprendimos que la verificación compara el producto contra lo que el equipo escribió, y el equipo tiende a escribir lo que ya pensó. Los cinco casos de verificación resultaron aprobados, pero tres de las cinco de validación encontraron fallas, específicamente, los casos de RF05 aprobados permitían saltarse reglas de negocio. Esto demuestra que una buena verificación no garantiza una validación efectiva si los criterios de aceptación están mal planteados.

11.2. Reflexión de Gonzalo

1. ¿Qué ambigüedad influyó más en el diseño y cómo la resolvieron? La ambigüedad A04 influyó en el diseño de la autenticación y las vistas del sistema. El enunciado no aclaraba si un usuario con rol de encargado podía también solicitar equipos. Como equipo, decidimos que por simplicidad era mejor separar la lógica y limitar a los encargados a solo administrar las solicitudes entrantes. En la vida real esto implicaría que un encargado que quiera pedir equipos

va a estar enrolado igualmente en el archivo de solicitantes. Esta decisión terminó ahorrando una vista extra para el encargado.

2. ¿Cuál fue la prueba más útil y qué riesgo permitió reducir? La prueba más útil fue CP-39, diseñada para detectar si la suite de pruebas intentaba conectarse al DSN real de Sentry al importar el módulo. Inicialmente fallaba porque la configuración de Sentry se inicializaba en tiempo de importación, lo que habría contaminado el panel de producción con eventos de prueba. Esto redujo el riesgo de que datos falsos ensuciaran el monitoreo de errores real, obligando a aislar completamente la configuración de Sentry en los tests mediante un DSN de prueba.
3. ¿Qué fallo o comportamiento inesperado detectaron y cómo lo corrigieron? Durante la ejecución de los casos CP-50 a CP-52, identificamos el defecto #29: el sistema aceptaba solicitudes con fechas ya vencidas, lo que dejaba el equipo inmovilizado en estado vencido y bloqueado para otros préstamos. Para corregirlo, agregamos una validación en el momento de crear la solicitud que impide seleccionar una fecha de préstamo anterior a la fecha actual, y escribimos el caso de prueba CP-52 que reproduce este escenario y lo marca como fallido hasta que se implemente la restricción. Este defecto había pasado desapercibido en los ocho casos de aceptación originales porque estos solo cubrían solicitudes con fechas futuras o actuales.
4. ¿Qué aporte de su compañero mejoró su trabajo? Incluir evidencia. Mi compañero Renato no solo encontró los defectos #27, #28 y #29 en la prueba cruzada de RF05, sino que escribió los casos de prueba CP-50, CP-51 y CP-52 que los reproducían, marcándolos como fallidos para que la suite los reportara. La evidencia está en la propia tabla de defectos del informe (Sección 8) y en el hecho de que, sin esos casos escritos por él, yo no habría podido corregir los fallos sin reproducirlos primero; la suite me habría mostrado un error genérico en lugar de un escenario concreto que pude depurar.
5. ¿Qué parte del sistema podría fallar todavía? El único riesgo residual es la falta de validación con usuarios reales, dado que se hicieron supuestos que redujeron la complejidad del problema pero que en la práctica pueden intervenir negativamente en el flujo de trabajo de los encargados de un laboratorio real.
6. ¿Qué aprendieron sobre la diferencia entre verificar y validar? Aprendí que la diferencia crítica está en que la verificación chequea si se construyó lo especificado (pasamos los 5 casos de verificación de RF01-RF05), pero la validación requiere probar si resuelve el problema real en su entorno de operación. Esto quedó claro al integrar con Sentry: mi código pasaba todas las verificaciones locales, pero al probar contra el panel real falló por problemas de configuración de logging que no se veían en unit tests, demostrando que pasar verificaciones no garantiza que el sistema funcione en su entorno real de operación.

11.3. Conclusiones

El resultado más claro de este trabajo no es la cantidad de código escrito sino dónde aparecieron los problemas. Las cinco actividades de verificación resultaron aprobadas; tres de las cinco de validación encontraron fallas. Los casos de aceptación de RF05 pasaron todos, y aun así el requerimiento permitía violar dos reglas centrales del enunciado. Eso deja una lección que atraviesa el proyecto: un criterio de aceptación mal planteado produce pruebas que pasan y software que no sirve.

11.4. Riesgos abiertos

- **Concurrencia.** La aplicación es de un solo proceso y no bloquea archivos. Dos instancias sobre el mismo directorio de datos podrían pisarse. Está declarado y no resuelto.

- **Utilización del laboratorio.** Bloquear una unidad desde que se pide, sin aprobar, reduce la utilización. Es el costo aceptado de no prometer fechas de liberación que no se pueden cumplir.